

# ORDER API

Backoffice et API REST headless de gestion e-commerce sécurisé

---

*Documentation du projet - Planification des Sprints*

Version 0.3.0 - 2026

# 1. Définition du problème

---

Aujourd'hui, les petites et moyennes entreprises (PME) qui lancent leur e-commerce n'ont pas toujours les ressources pour développer un système sur mesure. Elles se tournent vers des solutions clés en main comme Shopify, WooCommerce ou PrestaShop pour leur rapidité de mise en place. Mais ces outils atteignent rapidement leurs limites dès que l'entreprise veut adapter la plateforme à ses processus spécifiques, connecter des outils tiers ou exploiter finement ses données métier.

Cette dépendance entraîne trois problèmes concrets :

- Rigidité technique : impossible d'adapter les flux métier sans passer par un développeur et payer des frais de personnalisation élevés.
- Perte de souveraineté sur les données : les données clients, commandes et performances sont hébergées chez un tiers, sans accès direct ni export structuré.
- Absence de pilotage métier : sans dashboards ni exports automatisés, les équipes prennent des décisions à l'aveugle, sans indicateurs fiables ni temps réel.

Résultat : l'entreprise reste tributaire d'un outil conçu pour le grand nombre, pas pour ses besoins spécifiques. Elle ne peut ni évoluer librement, ni analyser ses données, ni intégrer ses propres outils sans friction.

## 1.1 Glossaire des concepts clés

---

### **Backoffice**

Interface ou système de gestion interne d'une entreprise, invisible pour les clients finaux. C'est l'outil utilisé par les équipes pour gérer les commandes, les produits, les utilisateurs et les données. Par opposition au "front-end" (la boutique visible par le client), le backoffice est le moteur qui fait tourner les opérations.

### **API REST headless**

Une API (Application Programming Interface) est un point d'accès standardisé qui permet à des systèmes de communiquer entre eux. REST désigne un style d'architecture basé sur des adresses web claires (ex. : GET /orders pour lister les commandes). Headless signifie que le système ne fournit pas d'interface visuelle intégrée : il expose uniquement des données et des actions, que n'importe quelle interface (site web, application mobile, outil tiers) peut consommer librement. C'est ce qui donne toute la flexibilité du système.

### **RBAC (Role-Based Access Control)**

Système de contrôle d'accès basé sur les rôles. Chaque utilisateur du système se voit attribuer un rôle (admin, manager, commercial, opérationnel) qui détermine précisément ce qu'il peut voir et faire. Un commercial peut consulter les commandes mais pas modifier le catalogue. Un opérationnel peut lire les dashboards mais pas exporter les données clients. Cette granularité garantit la sécurité des accès et la confidentialité des données.

### **Païement idempotent**

Un paiement est dit idempotent lorsqu'il peut être soumis plusieurs fois sans produire d'effet supplémentaire. Concrètement : si un client clique deux fois sur "Payer" ou si le réseau renvoie la requête en double, la commande n'est débitée qu'une seule fois. Ce comportement est garanti par une clé unique (Idempotency-Key) transmise avec chaque requête de paiement et mémorisée dans Redis.

## 1.2 Personas

---

Trois profils utilisateurs ont été identifiés comme cibles principales de la plateforme. Chaque profil a des besoins, des contraintes et un niveau de maîtrise technique différent.

### **Persona 1 - Le fondateur de PME e-commerce**

- Profil : Dirigée une boutique en ligne de 10 à 50 commandes par jour. Non technique.
- Besoin : suivre les commandes et les paiements sans dépendre d'un développeur pour chaque export de données.
- Frustration actuelle : Shopify lui coûte cher en abonnement et en apps tierces, et il ne peut pas accéder librement à ses données brutes.
- Ce qu'il attend : un backoffice simple, des exports CSV en un clic, des dashboards lisibles sans formation.

## Persona 2 - Le responsable opérationnel

- Profil : Gère une équipe de 3 à 10 personnes (commerciaux, logistique). Maîtrise d'Excel, pas de code.
- Besoin : visualiser les commandes en cours, suivre les expéditions, détecter les incidents en temps réel.
- Frustration actuelle : les outils existants mélangent tout, il doit naviguer entre plusieurs onglets pour avoir une vue complète de l'activité.
- Ce qu'il attend : un dashboard unifié avec accès sécurisé par rôle, et des alertes automatiques en cas d'anomalie.

## Persona 3 - Le développeur intégrateur

- Profil : Développeur freelance ou en agence, chargé d'intégrer un backend e-commerce pour un client PME.
- Besoin : une API bien documentée, des endpoints stables, une authentification robuste et une architecture extensible.
- Frustration actuelle : les solutions SaaS imposées n'exposent pas de vraie API REST, ou le font de manière partielle et mal documentée.
- Ce qu'il attend : un projet open source, bien structuré, avec documentation Swagger automatique et tests couvrant les cas critiques.

## 1.3 Proposition de valeur et positionnement marché

Order API n'est pas une alternative à Shopify pour vendre en ligne. C'est un système de gestion opérationnelle et analytique, conçu pour les équipes qui ont besoin de contrôle, de flexibilité et d'accès direct à leurs données métier. L'idée centrale : une architecture headless API-first qui peut se connecter à n'importe quel frontend ou outil tiers, sans imposer une stack rigide.

### Comparaison avec les solutions existantes

Le tableau ci-dessous positionne Order API par rapport aux solutions e-commerce les plus répandues sur le marché.

| Critère            | Shopify     | WooCommerce | Medusa.js | Sur mesure   | Order API     |
|--------------------|-------------|-------------|-----------|--------------|---------------|
| API REST headless  | Partiel     | Non         | Oui       | Variable     | Oui - natif   |
| RBAC granulaire    | Non         | Partiel     | Oui       | Variable     | Oui - natif   |
| Exports CSV/Excel  | App payante | Plugin      | Partiel   | à développer | Oui - roadmap |
| Monitoring intégré | Non         | Non         | Partiel   | à développer | Oui - natif   |
| Open source        | Non         | Oui         | Oui       | Oui          | Oui           |
| Coût initial       | Abonnement  | Hébergement | Gratuit   | Élevé        | Gratuit       |

*Order API se positionne comme une alternative open source, headless et orientée données, à destination des équipes techniques et des PME qui souhaitent reprendre le contrôle sur leur infrastructure e-commerce sans les contraintes des solutions SaaS propriétaires.*

## 2. Solution

La solution repose sur une architecture API-first.

Toute la logique du système est accessible via des API, on peut donc connecter facilement n'importe quel front ou outil externe et gérer le cycle de vie complet d'une commande, de manière sécurisée.

Sur la sécurité, on a mis en place un système de contrôle d'accès par rôle (RBAC) :

La plateforme est pensée pour trois profils métier :

- Administrateurs : gèrent le catalogue et les données, exportent et analysent l'activité
- Commerciaux : suivent les commandes, les paiements et la performance commerciale
- Opérationnels : supervisent l'activité en temps réel et détectent les incidents.

Chaque profil accède uniquement aux données pertinentes via un système de rôles sécurisé.

En parallèle : monitoring temps réel des perfs et des erreurs, pour détecter les problèmes rapidement.

Une solution fiable, sécurisée, capable de s'adapter à la croissance.

### 3. Vision du produit

---

Nous avons pris le problème à l'envers.

Plutôt qu'imposer un outil rigide, on est parti des vrais besoins des PME : flexibilité, simplicité, contrôle.

- API headless : liberté totale côté front
- Backoffice simple : utilisable par des équipes non techniques

L'idée : permettre aux entreprises de faire évoluer leur e-commerce librement, sans dépendre en permanence de développeurs, tout en restant pleinement souveraines sur leurs données.

On apporte la flexibilité du sur-mesure, sans la complexité technique qui va avec.

#### Objectifs de la vision

- La plateforme est conçue pour répondre aux besoins des administrateurs, commerciaux et opérationnels, en leur donnant un accès direct et adapté aux données métier.
- Permettre aux administrateurs un accès en libre-service aux exports de données en CSV et Excel.
- Fournir des tableaux de bord métier en temps réel.
- Automatiser les rapports KPI périodiques via des scripts Python planifiés.
- Mettre en place un pipeline de données automatisé (collecte, transformation, stockage).

#### Valeur ajoutée

- Meilleure gouvernance des données : les clients ne peuvent pas accéder aux données d'autres utilisateurs
- Efficacité opérationnelle : les administrateurs exportent les données sans accès direct à la base de données
- Visibilité métier : les dashboards Grafana mettent en avant des indicateurs actionnables
- Architecture scalable : le système RBAC est prêt à accueillir des rôles supplémentaires

Au-delà de l'API transactionnelle, ORDER API pose les bases d'une couche analytique : collecte automatisée des KPI, exports structurés et dashboards métier. C'est le premier maillon d'un pipeline de données orienté décision.

## 4. Rôles et permissions

---

| Rôle         | Profil                       | Permissions                                                                                           | Implémenté |
|--------------|------------------------------|-------------------------------------------------------------------------------------------------------|------------|
| User         | client                       | Passer des commandes, consulter ses propres commandes et factures                                     | v0.2.0     |
| Admin        | administrateur               | Gérer le catalogue, exporter les données, accéder aux dashboards, superviser l'ensemble de l'activité | v0.3.0     |
| Manager      | équipe métier                | Gérer le catalogue produit, les prix et le stock. Accès aux exports de données et aux KPI métier      | v0.3.0     |
| Commercial   | équipe métier                | Consulter les commandes, suivre les paiements, analyser la performance commerciale                    | Roadmap    |
| Opérationnel | équipe métier / exploitation | Superviser les métriques, surveiller les incidents, accéder aux dashboards temps réel                 | Roadmap    |

Les rôles Manager, Commercial et Opérationnel sont actuellement en cours d'implémentation (v0.3.0). Ils s'appuient sur le système `require_role()` existant et seront activés progressivement avec leurs permissions granulaires dédiées. En base de données, les valeurs techniques utilisées sont : manager, operator, viewer ; les noms affichés (Manager, Commercial, Opérationnel) correspondent aux profils métier.

## 5. Product Backlog

---

Dans ce backlog, deux types de user stories sont distingués :

- User stories métier :
  - Exprimées du point de vue des utilisateurs finaux (administrateurs, commerciaux, opérationnels).
  - Elles décrivent des besoins fonctionnels directement liés à l'usage du produit et à la création de valeur métier.
- User stories techniques :
  - Exprimées du point de vue du système ou des développeurs.
  - Elles concernent des besoins d'architecture, de sécurité, de performance ou de maintenabilité.
  - Elles sont indispensables au bon fonctionnement du produit mais ne sont pas directement visibles par les utilisateurs.

Cette distinction permet de mieux structurer le développement et de prioriser la valeur métier tout en garantissant la robustesse technique.

## EPIC 1 - Contrôle d'accès basé sur les rôles (RBAC)

| ID  | User Story                                                                                                                                                                    | Type      | Rôle    | Priorité | Story Points | Status  |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|--------------|---------|
| US1 | En tant qu'administrateur, je veux que les routes de création/modification/suppression des produits soient protégées afin que seuls les admins puissent modifier le catalogue | Métier    | admin   | Haute    | 5            | À faire |
| US2 | En tant que client, je veux accéder uniquement à mes propres commandes et factures afin que mes données restent privées                                                       | Métier    | user    | Haute    | 3            | À faire |
| US3 | En tant qu'administrateur, je veux consulter toutes les commandes et tous les utilisateurs afin d'avoir une visibilité opérationnelle complète                                | Métier    | admin   | Haute    | 3            | À faire |
| US4 | En tant que système, je veux centraliser la gestion des rôles afin de garantir un contrôle d'accès cohérent sur toutes les routes                                             | Technique | système | Haute    | 5            | À faire |

## EPIC 2 - Exports de données (CSV / Excel)

| ID  | User Story                                                                                                            | Type   | Rôle  | Priorité | Story Points | Status  |
|-----|-----------------------------------------------------------------------------------------------------------------------|--------|-------|----------|--------------|---------|
| US5 | En tant qu'administrateur, je veux exporter toutes les commandes en CSV afin de les analyser dans Excel ou Python     | Métier | admin | Haute    | 5            | À faire |
| US6 | En tant qu'administrateur, je veux exporter tous les produits en CSV afin d'auditer le catalogue                      | Métier | admin | Moyenne  | 3            | À faire |
| US7 | En tant qu'administrateur, je veux exporter tous les utilisateurs en CSV afin de gérer la base clients                | Métier | admin | Moyenne  | 3            | À faire |
| US8 | En tant qu'administrateur, je veux exporter toutes les factures en Excel (.xlsx) afin de les utiliser en comptabilité | Métier | admin | Moyenne  | 5            | À faire |
| US9 | En tant qu'administrateur, je veux filtrer les exports par plage de dates afin de cibler mon analyse                  | Métier | admin | Faible   | 2            | À faire |

## EPIC 3 - Tableaux de bord métier (Grafana)

| ID   | User Story                                                                                                                    | Type   | Rôle  | Priorité | Story Points | Status  |
|------|-------------------------------------------------------------------------------------------------------------------------------|--------|-------|----------|--------------|---------|
| US10 | En tant qu'administrateur, je veux visualiser les commandes par jour afin de suivre le volume métier                          | Métier | admin | Haute    | 3            | À faire |
| US11 | En tant qu'administrateur, je veux visualiser le temps de traitement moyen des commandes afin de détecter les ralentissements | Métier | admin | Haute    | 5            | À faire |
| US12 | En tant qu'administrateur, je veux visualiser des alertes quand le taux d'erreur dépasse 5% afin d'être notifié des incidents | Métier | admin | Moyenne  | 5            | À faire |
| US13 | En tant qu'administrateur, je veux consulter le chiffre d'affaires par jour afin de suivre la performance financière          | Métier | admin | Faible   | 3            | À faire |

## EPIC 4 - Pipeline de données & automatisation

### Objectif

Construire un pipeline de données automatisé permettant de calculer, stocker et exposer des KPI métier.

| ID   | User Story                                                                                                                                                               | Type                  | Rôle    | Priorité | Story Points | Status  |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------|----------|--------------|---------|
| US14 | En tant qu'administrateur, je veux un script Python qui calcule les KPI quotidiens (nombre de commandes, délai moyen, taux d'erreur) afin d'obtenir un résumé périodique | Métier (orienté data) | admin   | Haute    | 8            | À faire |
| US15 | En tant qu'administrateur, je veux que le script KPI s'exécute automatiquement chaque jour afin que le reporting ne nécessite aucune action manuelle                     | Technique             | admin   | Haute    | 5            | À faire |
| US16 | En tant qu'administrateur, je veux que le rapport KPI soit exporté en CSV afin de pouvoir le stocker et l'analyser dans le temps                                         | Métier                | admin   | Moyenne  | 3            | À faire |
| US17 | En tant que système, je veux stocker les KPI calculés dans une table dédiée afin de conserver un historique analytique interrogeable                                     | Technique             | système | Moyenne  | 5            | À faire |

## 6. Critères d'acceptation

### Objectif des critères

Les critères d'acceptation définissent les conditions de validation des User Stories.

Ils permettent de s'assurer que les fonctionnalités répondent aux besoins métier et sont testables.

### US1 - Protection des routes produits (RBAC admin)

| ID  | Scénario                             | Given (Étant donné)                    | When (Quand)                                | Then (Alors)                                                     |
|-----|--------------------------------------|----------------------------------------|---------------------------------------------|------------------------------------------------------------------|
| AC1 | Création refusée sans rôle admin     | Je suis authentifié avec le rôle user  | J'envoie POST /products                     | Je reçois une réponse 403 Forbidden                              |
| AC2 | Modification refusée sans rôle admin | Je suis authentifié avec le rôle user  | J'envoie PATCH /products/{id}               | Je reçois une réponse 403 Forbidden                              |
| AC3 | Suppression refusée sans rôle admin  | Je suis authentifié avec le rôle user  | J'envoie DELETE /products/{id}              | Je reçois une réponse 403 Forbidden                              |
| AC4 | Création autorisée pour l'admin      | Je suis authentifié avec le rôle admin | J'envoie POST /products avec un body valide | Le produit est créé et je reçois 201 avec les données du produit |
| AC5 | Accès non authentifié refusé         | Je ne suis pas authentifié             | J'envoie POST /products                     | Je reçois une réponse 401 Unauthorized                           |
| AC6 | Consultation publique autorisée      | Je ne suis pas authentifié             | J'envoie GET /products                      | Je reçois 200 avec la liste des produits                         |

## US5 - Export des commandes en CSV

| ID  | Scénario                            | Given (Étant donné)                              | When (Quand)                                                         | Then (Alors)                                                                    |
|-----|-------------------------------------|--------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| AC1 | Export refusé sans rôle admin       | Je suis authentifié avec le rôle user            | J'envoie GET /exports/orders                                         | Je reçois une réponse 403 Forbidden                                             |
| AC2 | Export refusé sans authentification | Je ne suis pas authentifié                       | J'envoie GET /exports/orders                                         | Je reçois une réponse 401 Unauthorized                                          |
| AC3 | Export CSV généré pour l'admin      | Je suis authentifié avec le rôle admin           | J'envoie GET /exports/orders                                         | Je reçois un fichier CSV téléchargeable avec l'en-tête Content-Type: text/csv   |
| AC4 | Contenu du fichier correct          | Des commandes existent en base                   | Le fichier CSV est généré                                            | Il contient les colonnes id, user_id, status, total_cents, currency, created_at |
| AC5 | Fichier vide si aucune commande     | Aucune commande n'existe en base                 | J'envoie GET /exports/orders                                         | Je reçois un fichier CSV avec uniquement la ligne d'en-tête                     |
| AC6 | Filtre par date fonctionnel         | Je suis administrateur et des commandes existent | J'envoie GET /exports/orders?from_date=2026-01-01&to_date=2026-01-31 | Le fichier ne contient que les commandes de la période demandée                 |

## US10 - Visualisation des commandes par jour

| ID  | Scénario               | Given (Étant donné)                        | When (Quand)                    | Then (Alors)                                                   |
|-----|------------------------|--------------------------------------------|---------------------------------|----------------------------------------------------------------|
| AC1 | Affichage du graphique | Je suis un administrateur authentifié      | J'accède au dashboard Grafana   | Je vois un graphique affichant le nombre de commandes par jour |
| AC2 | Exactitude des données | Il existe des commandes en base de données | Le graphique est affiché        | Les données correspondent aux commandes enregistrées           |
| AC3 | Filtrage par date      | Je sélectionne une plage de dates          | Le Dashboard est mis à jour     | Seules les commandes de la période sélectionnée sont affichées |
| AC4 | Sécurité d'accès       | Je ne suis pas administrateur              | J'essaie d'accéder au Dashboard | L'accès est refusé                                             |
| AC5 | Temps réel             | Les données sont collectées                | Le Dashboard est affiché        | Les données sont mises à jour en quasi-temps réel (< 1 min)    |



## US14 - Script KPI quotidien

| ID  | Scénario                      | Given (Étant donné)                               | When (Quand)                          | Then (Alors)                                                              |
|-----|-------------------------------|---------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------|
| AC1 | Calcul du nombre de commandes | Des commandes existent en base pour la journée    | Le script kpi_report.py s'exécute     | Le rapport contient le nombre exact de commandes du jour                  |
| AC2 | Calcul du délai moyen         | Des commandes payées existent avec des timestamps | Le script s'exécute                   | Le rapport contient le délai moyen en secondes entre création et paiement |
| AC3 | Calcul du taux d'erreur       | Des requêtes HTTP ont été enregistrées            | Le script s'exécute                   | Le rapport contient le pourcentage de réponses 5xx sur le total           |
| AC4 | Export CSV généré             | Le script s'exécute sans erreur                   | L'exécution se termine                | Un fichier kpi_YYYY-MM-DD.csv est créé dans le dossier reports/           |
| AC5 | Exécution automatique à 06h00 | APScheduler est démarré avec l'API                | Il est 06h00                          | Le script s'exécute automatiquement sans intervention manuelle            |
| AC6 | Idempotence du script         | Le script a déjà été exécuté aujourd'hui          | Le script s'exécute une deuxième fois | Le fichier CSV existant est écrasé sans erreur                            |

## 7. Planification des Sprints (3 semaines)

### SPRINT 1 (Semaine 1) : RBAC et contrôle d'accès

#### Objectif du sprint

Appliquer le contrôle d'accès basé sur les rôles sur toutes les routes existantes. Les utilisateurs n'accèdent qu'à leurs propres données ; les administrateurs ont un accès complet.

#### User Stories

- US1 : Protéger les routes d'écriture des produits (POST/PUT/PATCH/DELETE) avec `require_admin`
- US2 : Filtrer les commandes et factures par utilisateur authentifié (rôle user)
- US3 : Permettre aux administrateurs de lister toutes les commandes et tous les utilisateurs
- US4 : Implémenter une dépendance `require_role()` réutilisable

#### Tâches techniques

| ID | Tâche                                                                                                                  | US liée   | Fichier(s)                                | Est. |
|----|------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------|------|
| T1 | Créer la fonction <code>factory require_role(role)</code>                                                              | US4       | <code>app/core/auth.py</code>             | 1h   |
| T2 | Protéger <code>POST/PUT/DELETE /products</code> avec <code>require_admin</code>                                        | US1       | <code>app/api/products.py</code>          | 1h   |
| T3 | Filtrer <code>GET /orders</code> pour ne retourner que les commandes de l'utilisateur connecté                         | US2       | <code>app/api/orders.py</code>            | 2h   |
| T4 | Filtrer <code>GET /invoices</code> pour ne retourner que les factures de l'utilisateur connecté                        | US2       | <code>app/api/invoices.py</code>          | 1h   |
| T5 | Ajouter <code>GET /admin/orders</code> (toutes les commandes) et <code>GET /admin/users</code> (tous les utilisateurs) | US3       | <code>app/api/admin.py</code> (nouveau)   | 2h   |
| T6 | Écrire les tests pour chaque rôle (accès user, accès admin, accès interdit)                                            | US1, 2, 3 | <code>tests/test_rbac.py</code> (nouveau) | 3h   |

#### Livrables

- Dépendance `require_role()` fonctionnelle et testée
- Toutes les routes protégées selon le tableau des rôles
- Nouveau fichier de tests : `tests/test_rbac.py`
- Tous les tests existants toujours verts (CI green)

## SPRINT 2 (Semaine 2) : Exports de données (CSV / Excel)

### Objectif du sprint

Exposer des endpoints d'export sécurisés pour les admins. Tous les exports nécessitent une authentification avec le rôle admin et retournent des fichiers téléchargeables.

### User Stories

- US5 : Export des commandes en CSV
- US6 : Export des produits en CSV
- US7 : Export des utilisateurs en CSV
- US8 : Export des factures en Excel (.xlsx)
- US9 : Filtre optionnel par plage de dates sur les exports

### Tâches techniques

| ID  | Tâche                                                                                            | US liée         | Fichier(s)                      | Est.  |
|-----|--------------------------------------------------------------------------------------------------|-----------------|---------------------------------|-------|
| T7  | Ajouter openpyxl dans requirements.txt                                                           | US8             | requirements.txt                | 15min |
| T8  | Créer app/api/exports.py avec GET /exports/orders (CSV)                                          | US5             | app/api/exports.py (nouveau)    | 2h    |
| T9  | Ajouter GET /exports/products et GET /exports/users (CSV)                                        | US6, 7          | app/api/exports.py              | 1h    |
| T10 | Ajouter GET /exports/invoices (.xlsx avec openpyxl)                                              | US8             | app/api/exports.py              | 2h    |
| T11 | Ajouter les paramètres optionnels ?from_date=&to_date= sur l'export des commandes (filtre dates) | US9             | app/api/exports.py              | 1h    |
| T12 | Enregistrer le router exports dans app/main.py                                                   | US5, 6, 7, 8, 9 | app/main.py                     | 15min |
| T13 | Écrire les tests pour les routes d'export (auth, format, contenu)                                | US5, 6, 7, 8, 9 | tests/test_exports.py (nouveau) | 2h    |

### Livrables

- 4 endpoints d'export actifs et protégés par require\_admin
- Exports CSV téléchargeables depuis GET /exports/orders, /products, /users
- Export Excel téléchargeable depuis GET /exports/invoices
- Nouveau fichier de tests : tests/test\_exports.py

## SPRINT 3 (Semaine 3) : Dashboards Grafana

### Objectif du sprint

Mettre en place les dashboards métiers pour le suivi des performances.

### User Stories

- US10 : Commandes par jour
- US11 : Temps de traitement moyen
- US12 : Alertes taux d'erreur
- US13 : Chiffre d'affaires par jour

### Tâches techniques

| ID  | Tâche                                                                     | US liée      | Fichier(s)                                   | Est. |
|-----|---------------------------------------------------------------------------|--------------|----------------------------------------------|------|
| T14 | Ajouter des métriques Prometheus (orders_total, order_processing_seconds) | US10, 11, 12 | app/core/metrics/prometheus.py               | 2h   |
| T15 | Créer un panel Grafana : commandes/jour (bar chart)                       | US10         | monitoring/grafana/dashboards/order_api.json | 1h   |
| T16 | Créer un panel Grafana temps réel de traitement (jauge)                   | US11         | monitoring/grafana/dashboards/order_api.json | 1h   |
| T17 | Configurer alerte taux d'erreur                                           | US12         | monitoring/grafana/dashboards/order_api.json | 1h   |

### Livrables

- 3 nouveaux panels Grafana (commandes/jour, temps de traitement moyen, alerte taux d'erreur)
- Tous les tests existants toujours verts - CI green

## SPRINT 4 (Semaine 4) : Automatisation KPI

### Objectif du sprint

Automatiser la génération et l'export des indicateurs métiers.

### User Stories

- US14 : Calcul des KPI
- US15 : Exécution automatique
- US16 : Export des KPI

### Tâches techniques

| ID  | Tâche                                                                                                | US liée  | Fichier(s)                                | Est. |
|-----|------------------------------------------------------------------------------------------------------|----------|-------------------------------------------|------|
| T18 | Écrire scripts/kpi_report.py : récupérer les données depuis la DB, calculer les KPI, exporter en CSV | US14     | scripts/kpi_report.py (nouveau)           | 3h   |
| T19 | Intégrer APScheduler dans FastAPI pour exécuter kpi_report.py tous les jours à 06h00                 | US15     | app/main.py ou app/core/scheduler.py      | 2h   |
| T20 | Ajouter APScheduler dans requirements.txt et tester le job planifié                                  | US15, 16 | requirements.txt, tests/test_scheduler.py | 1h   |

### Livrables

- scripts/kpi\_report.py produisant un CSV quotidien
- APScheduler exécutant le script automatiquement à 06h00
- Tous les tests verts - CI green

## 8. Architecture technique

| Couche                                       | Technologie                                                      | Rôle                                                      |
|----------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------|
| API                                          | FastAPI 0.116                                                    | Endpoints REST, authentification JWT, middleware RBAC     |
| Base de données                              | PostgreSQL 15                                                    | Commandes, produits, utilisateurs, factures               |
| Cache / Rate limiting                        | Redis 7                                                          | Liste noire JWT, rate limiting, idempotence               |
| Métriques                                    | prometheus-client 0.22                                           | Exposition de /metrics pour Prometheus                    |
| Monitoring                                   | Prometheus + Grafana 11.4                                        | Collecte des métriques et dashboards                      |
| Exports                                      | openpyxl + csv (stdlib)                                          | Génération de fichiers CSV et Excel                       |
| Automatisation                               | APScheduler                                                      | Jobs KPI planifiés intégrés dans FastAPI                  |
| Intégration continue                         | GitHub Actions                                                   | Suite de tests automatisée à chaque push                  |
| Pipeline de données                          | Pipeline ETL léger (Extract-Transform-Load)                      | Collecte, transformation et agrégation des données métier |
| Stockage analytique                          | PostgreSQL (schéma analytics ou tables dédiées)                  | Stockage historique des KPI (revenu, commandes, erreurs)  |
| Vers une séparation OLTP / OLAP (simplifiée) | Base transactionnelle isolée des requêtes analytiques (roadmap)" | Optimisation des requêtes de reporting et dashboards      |

## 9. État actuel (base v0.3.0)

| Fonctionnalité                                   | État         | Notes                                                                                                                |
|--------------------------------------------------|--------------|----------------------------------------------------------------------------------------------------------------------|
| Auth (register, login, logout, refresh)          | Fait         | Tokens JWT accès + refresh                                                                                           |
| users/me                                         | Fait         | Retourne le profil de l'utilisateur connecté                                                                         |
| RBAC require_role () + filtrage ownership        | Fait         | require_role(), orders et invoices filtrés par user                                                                  |
| CRUD Produits                                    | Fait         | GET/POST/PUT/DELETE                                                                                                  |
| Commandes (création, articles, paiement, statut) | Fait         | Filtrées par utilisateurs authentifiés (ownership)                                                                   |
| Factures                                         | Fait         | Créées automatiquement après paiement                                                                                |
| Paiement                                         | Fait         | Idempotence_Key (Redis)                                                                                              |
| Rate limiting (Redis)                            | Fait         | Par IP et par famille de routes                                                                                      |
| Middleware d'audit                               | Fait         | Écrit audit_log.csv                                                                                                  |
| Prometheus /metrics                              | Fait         | http_requests_total + histogramme de durée                                                                           |
| Dashboard Grafana                                | Fait         | Latency p95, RPS, taux d'erreur                                                                                      |
| Tests                                            | Faits        | 25 tests pytest async (auth, orders, products, invoices, customers) - stack migrée vers AsyncSession + AsyncClient   |
| Séparation Customers / Users                     | Fait         | Table customers avec FK UUID vers orders et invoices. Modèle, schemas, service et endpoints CRUD dédiés (v0.3.0)     |
| Gestion de stock (réservation)                   | Fait         | stock_quantity (stock physique) et reserved_quantity (commandes non payées). Décrément au paiement confirmé (v0.3.0) |
| Migration stack async (SQLAlchemy + endpoints)   | Fait         | AsyncSession, create_async_engine, lazy='selectin' sur toutes les relations. 14 fichiers migrés (v0.3.0)             |
| Routes admin (GET /admin/orders, /admin/users)   | En cours     | T5 - Sprint 1 non terminé                                                                                            |
| Exports de données                               | Non commencé | Sprint 2                                                                                                             |
| Tableaux de bord métier                          | Non commencé | Sprint 3                                                                                                             |
| Automatisation / scheduler                       | Non commencé | Sprint 4                                                                                                             |
| Pipeline de données (KPI, exports, dashboards)   | Non commencé | Structuration prévue - Sprint 2 à 4                                                                                  |